

SatQuery Architecture

Satellite Image Question Answering: Design Rationale and Implementation Blueprint

Contents

1. What SatQuery is Actually Trying to Solve	4
2. Why This Architecture Was Chosen	5
2.1 The Biggest Correction: 19 Classes vs. 44 Classes	5
3. The Central Architectural Philosophy	6
4. Why Symbolic Computation is So Important	6
5. Why the Previous Architecture Needed Correction	6
6. The Final Architecture at a High Level	7
7. Stage 1 — Input Validation	9
8. Stage 2 — Preprocessing	9
9. Optional Spectral Indices	9
10. Band-Presence Mask	9
11. Stage 3 — Query Understanding	9
12. Rule-Based Parser First	10
13. M10 — Query Classifier Fallback	10
14. Stage 4 — Deterministic Router	10
15. Why Deterministic Routing?	10
16. Stage 5 — Perception	10
17. Scene Cache	10
18. M1 — Multi-Sensor LULC Segmentation	11
19. Why Dual Encoder?	11

20. M1 Architecture	11
21. Why U-Net?	11
22. 44-Class Output + Auxiliary Heads	12
23. M1 Loss Function	12
24. Lovász-Softmax	12
25. Hierarchy-Aware Loss	12
26. Scale Consistency Loss	12
27. Why M1 is Trained From Scratch	12
28. M1 Training Dataset	13
29. M2 — Symbolic Geometry Engine	13
30. M2 Step 1 — Hierarchy Aggregation	13
31. M2 Step 2 — Binary Mask	13
32. M2 Step 3 — Morphological Cleanup	13
33. M2 Step 4 — Connected Components	13
34. Connectivity: 4 vs. 8	13
35. Minimum Mapping Unit (MMU)	14
36. Region Properties	14
37–41. Geometric Task Computations	14
42–43. Referring Expression and Point Detection	14
44. Caption Attributes	14
45. M3 — Binary VQA Decision Head	15
46. Why Not Just Use a Fixed Tolerance?	15
47. M3 Features	15
48. Why Train M3 on Predicted Maps?	15
49–51. M4 — MCQ Option Scorer	15
52–55. M5 — Scene Metadata Classifier	15
56–59. M6 — Siamese Semantic Change Model	16

60. Why SECOND?	16
61. M6 Training	16
62. M6 Loss	17
63–65. M7 — InternVL3-1B	17
66. M7 Training Curriculum	17
67. M7 Decoding Strategy	17
68. A Note on the Published 34.04 BLEU	18
69–72. M8 — Template-to-Style Caption Rewriter	18
73–75. M9 — Confidence Calibrator	18
76–77. Stage 8 — Abstention	18
78. Stage 9 — Answer Assembler	19
79. Final Outputs	19
80–81. How All Models Communicate	19
82. Datasets — The Complete Picture	19
83. Dataset 1 — reBEN	20
84. Dataset 2 — BigEarthNet.txt	20
85–86. Datasets 3 & 4 — SECOND and CDVQA	20
87–88. Datasets 5 & 6 — VRSBench and RSVQA	20
89–92. Indian Data	20
93–95. BHARAT-EO, BHARAT-TRAIN, BHARAT-VAL	21
96. Indian Domain Adaptation	21
97. Why Not LoRA for M1?	21
98. Why India is the Strongest Argument for This Architecture	21
99–100. Data Augmentation Strategy	21
101. Class Balancing	22
102–103. Validation Methodology	22
104–106. Accuracy is Not Guessed	22

107–108. Error Analysis	22
109–111. Baselines and Statistical Significance	23
112–113. Architecture A vs. B vs. C	23
114. Final Implementation Blueprint	23
115. Exact Model Summary	24
116. Pretraining Strategy — Memorize This Distinction	24
117. The 8-Week Execution Plan	25
118. The Three Numbers That Decide Whether the Project is Working	25
119. Published Reference Numbers	25
120. Why Adjacency is Such a Powerful Demonstration	26
121–122. The Two Biggest Limitations	26
123. What Makes Architecture B Convincing	26
124–129. End-to-End Examples	26
130. The Architecture in One Sentence	27
131. If Your Team Asks “Why So Many Models?”	27
132. Fifteen Facts to Memorize	27
133. The Most Important Conceptual Distinction	28

1. What SatQuery is Actually Trying to Solve

The system receives:

- one or two satellite images, usually GeoTIFFs
- a natural-language question

For example:

- “How much area is covered by forest?”
- “Are water bodies adjacent to urban areas?”
- “Which region is north of the agricultural area?”
- “What changed between image 1 and image 2?”

The naive approach would be:

Satellite image + question
 |
 VLM

|
answer

The architecture says don't do that for questions that can be calculated. Instead:

Satellite image
|
Land-cover segmentation
|
Pixel-level map
|
Geometry / spatial calculations
|
Exact numerical result
|
Answer

And only for language-heavy tasks:

Satellite image
|
VLM
|
Caption / free-form language

The reason is that the benchmark's ground truth itself was generated from pixel-level maps. Presence, count, size, adjacency, etc. are measurements extracted from those maps. Therefore, reproducing the same process is fundamentally aligned with how the benchmark was constructed.

2. Why This Architecture Was Chosen

The document first reviews the previous architecture and identifies several important problems.

2.1 The Biggest Correction: 19 Classes vs. 44 Classes

This is probably the single most important technical correction in the whole document.

The previous architecture apparently assumed:

Segmentation -> 19 classes

That is wrong for the pixel-level reference maps. The architecture establishes that:

CORINE Level 3 -> 44 classes

are used in the pixel-level reference maps. The 19-class vocabulary belongs to a different image-level multi-label representation.

Why does that matter? Suppose the map contains “continuous urban fabric” and “discontinuous urban fabric” — these are separate Level-3 classes. But a question could ask about “artificial surfaces,” which is a higher-level category. If you immediately reduce everything to 19 classes, you lose the detailed distinctions.

Therefore the architecture does:

```

Predict detailed L3 map
  |
  44 classes
  |
Question asks for some CLC level
  |
  Hierarchy mapping
  |
Aggregate required classes
  |
  Perform geometry

```

This is why the new architecture introduces hierarchy-aware aggregation.

3. The Central Architectural Philosophy

The document's central bet is:

Predict what needs perception; calculate what can be calculated.

For example, question: "How many forest regions are present?" The VLM does not need to answer this. Instead:

```

Satellite image -> M1 segmentation -> Forest pixels -> Binary forest mask
-> Connected components -> Number of components -> Answer = 7

```

Likewise, question: "What is the area of forest?"

```

Forest mask -> Count forest pixels -> pixel count x GSD^2 -> area

```

Question: "Are forest and water adjacent?"

```

Forest mask -> Dilate -> Intersect with water mask -> intersection exists? -> YES/NO

```

This is why the document calls the geometry engine deterministic. There is no neural network deciding whether two regions touch.

4. Why Symbolic Computation is So Important

The document makes a particularly strong observation. A reported frontier model with around 2 trillion parameters obtained only 55.86% on binary adjacency. But adjacency can be calculated with a simple morphological operation:

```

binary_dilation(A) intersect B

```

If the intersection is non-empty, adjacent = TRUE. So the architecture is essentially asking: why spend billions of parameters learning a relationship that can be measured exactly once we have the correct map? That is the scientific motivation behind the architecture.

5. Why the Previous Architecture Needed Correction

The review identifies twelve major issues. The important ones are:

F1 — Taxonomy error. 19-class segmentation was incorrect. Solution: 44-class L3 segmentation + hierarchy aggregation.

F2 — Wrong assumptions about accuracy. The previous architecture assumed counting >85%, adjacency >85%, area >80% before measuring the actual ceiling. The new architecture says: first run the symbolic engine on the ground-truth maps. That gives the oracle accuracy. Only then can you know how good the method can theoretically become.

F3 — Binary VQA isn’t simply exact comparison. Suppose the question says “Is there approximately 45,000 m² of forest?” The answer isn’t necessarily `computed == 45000`. The benchmark deliberately creates near-miss “NO” answers, so the system needs to learn the decision boundary. That’s M3.

F4 — MCQ metadata was being ignored. MCQ has eight categories: presence, area, count, adjacency, relative position, country, season, climate zone. The first five are geometric; the last three aren’t. So the architecture introduces M5.

F5 — Don’t force the same encoder into everything. The previous architecture wanted to share segmentation and VLM representations. The review rejects that because the segmentation task needs spatial precision, while the VLM’s pretrained ViTs were designed primarily for scene classification.

F6/F7 — Indian adaptation. The Indian domain shift is divided into: (A) spectral shift, (B) spatial/GSD shift, (C) SAR characteristics, (D) geography shift. Three can be simulated to some extent. Geographic shift can actually be measured using Sentinel-1/2 over India. Therefore the architecture creates BHARAT-EO / BHARAT-VAL.

F8 — Blind baselines. The system must also answer: what if I don’t use the image at all? Hence question-only baseline, majority baseline, class-prior baseline. Otherwise you don’t know whether the benchmark is actually testing vision.

F9 — Previous architecture was too big. Six students, eight weeks, sixteen components — too much. Architecture B cuts unnecessary components.

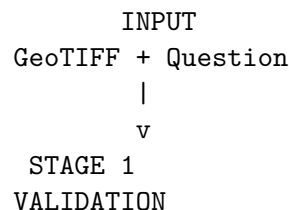
F10 — Don’t use an LLM for an 8-class router. Instead: TF-IDF → Linear SVM → 8 task classes. Cheap and auditable.

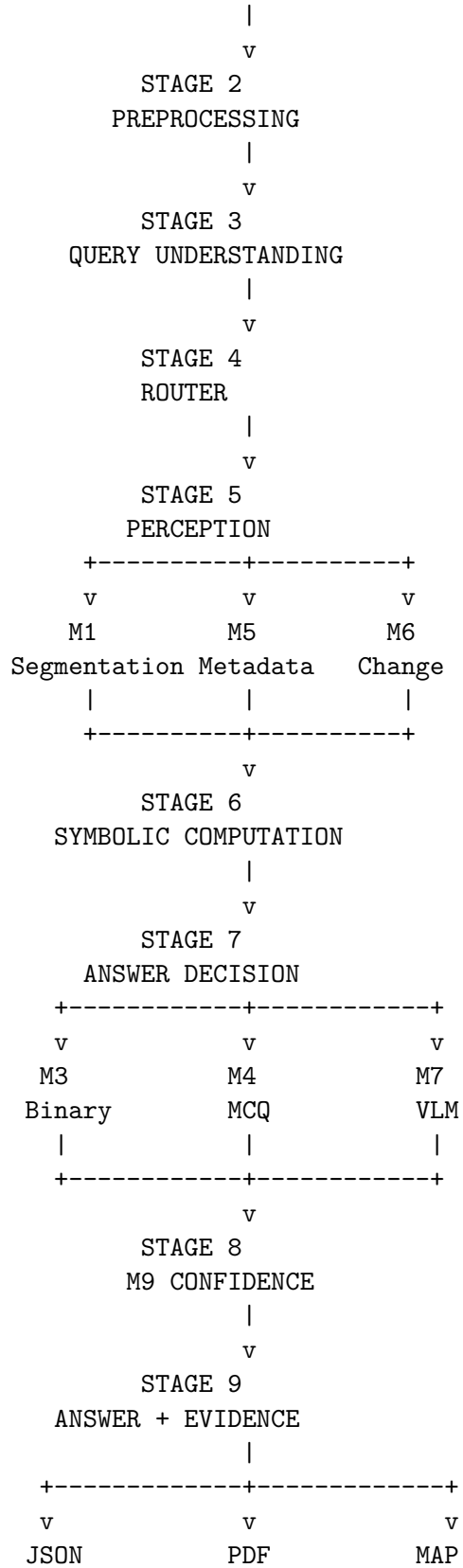
F11 — Geographic leakage. Randomly splitting satellite images can be dangerous because nearby images are correlated. Therefore validation is geographically blocked.

F12 — Confidence. Three raw confidence signals aren’t enough. They must be combined into an actual model: signals → logistic regression → P(answer correct).

These corrections produce Architecture B.

6. The Final Architecture at a High Level





7. Stage 1 — Input Validation

The system first receives 1 or 2 GeoTIFFs + text query. The validator is called V1. Technology: **rasterio**. It checks: number of bands, datatype, CRS, geotransform, image shape, NoData values, modality, whether two images are co-registered, and metadata strings. Then it creates an **InputManifest**, or rejects the input with a typed error.

Why this is necessary. A GeoTIFF contains spatial information (e.g., width = 120, height = 120, bands = 12, CRS, pixel size = 10m). If you accidentally load it using a normal image library, you can lose bands or geospatial information. That is why the architecture explicitly says rasterio only — not PIL, OpenCV, or generic imread — because these can mishandle multispectral and geospatial metadata.

8. Stage 2 — Preprocessing

After validation comes sensor preprocessing. There are two primary sensors:

Sentinel-1 (SAR). Two channels: VV, VH. The architecture converts linear power $\rightarrow$ dB $\rightarrow$ z-score normalization, using statistics calculated from the training set.

Sentinel-2. The architecture uses B02, B03, B04, B05, B06, B07, B08, B8A, B11, B12 — 10 bands. The native resolutions differ: 10m bands stay at 10m; 20m bands are bilinearly interpolated to 10m; 60m bands are discarded.

So: S1 = 2 channels, S2 = 10 channels, Total = 12 channels.

9. Optional Spectral Indices

Three optional derived features are mentioned:

- **NDVI** (Normalized Difference Vegetation Index) — used mainly for vegetation information.
- **NDWI** (Normalized Difference Water Index) — useful for water-related information.
- **NDBI** (Normalized Difference Built-up Index) — useful for built-up regions.

These are not considered absolutely necessary when all raw spectral bands are available, but they become useful during band dropout. For example, if SWIR is unavailable, NDBI/other relationships can preserve some useful information. The architecture therefore says to test them through ablation.

10. Band-Presence Mask

Suppose SWIR is missing. You could simply set $SWIR = 0$, but the model cannot know whether 0 means a genuinely dark surface or a sensor band that was unavailable — those are completely different meanings. So the architecture adds a band-presence mask of length 10, which tells the model whether a band exists or was dropped. This is important for the Indian sensor-robustness strategy.

11. Stage 3 — Query Understanding

This is Q1. Example: “How many water regions are there?” becomes `intent = COUNT, class = WATER, qualifier = none`. Another: “What is the area of artificial surfaces?” becomes `intent`

= AREA, class = ARTIFICIAL_SURFACES, level = appropriate CLC level. Another: “Which region is north of the forest?” might become intent = RELATIVE_POSITION, class_A = region, class_B = forest, qualifier = north_of.

12. Rule-Based Parser First

The architecture does not immediately call an LLM. It first uses closed template rules because the benchmark has a closed vocabulary. This gives intent, class A, class B, CLC level, and qualifier. The CLC synonym table resolves different words (urban, urban fabric, built-up, artificial surface) to the same class depending on the question.

13. M10 — Query Classifier Fallback

If the rules fail: Query → TF-IDF → Linear SVM → 8-way classification. This is M10. The eight task categories correspond to the query types handled by the architecture. Why SVM? Because this is a classic “which of these eight known task types does this question belong to?” classification problem — not open-ended language generation. It runs on CPU and is easy to audit.

14. Stage 4 — Deterministic Router

Once the question is understood, the router (R1) decides which components need to execute, using (intent x input configuration) → tool plan. Examples:

- **Area question:** AREA → M1 → M2 → M3 if necessary → M9 → answer
- **Metadata MCQ:** MCQ + country → M5 → M4 → M9 → answer
- **Caption:** CAPTION → M1/M2 → caption attributes → M8 or M7
- **Change question:** Two images → M6 → transition matrix → M2 → answer

The router is deterministic. No LLM decides which tool to call, no eval(), no dynamic plugin loading. The allowed tools and parameters are frozen and validated using Pydantic.

15. Why Deterministic Routing?

Because this system needs to be reproducible, auditable, secure, and predictable. If an LLM decided the execution path dynamically, you would have an unpredictable trace. Instead, a known rule like intent = AREA → AREA → M1 → M2 → M3/M9 → assembler runs the same way every time, making the execution trace meaningful.

16. Stage 5 — Perception

This is where the actual learned visual models operate: M1 (LULC segmentation), M5 (metadata classification), M6 (change detection). There is also a scene cache.

17. Scene Cache

Suppose the user asks three questions in a row about the same image. You don’t want to run segmentation three times. Instead: Image → M1 → land-cover map → CACHE. Every subsequent

question uses the same map. The cache is keyed using `hash(scene bytes) + model version`, so a map produced by a different model checkpoint isn't accidentally reused. This is both a performance optimization and a reproducibility mechanism.

18. M1 — Multi-Sensor LULC Segmentation

This is the most important model in the whole architecture. Its job: convert Sentinel-1 + Sentinel-2 into a pixel-level land-cover map.

- Input: S1 = 2 channels, S2 = 10 channels
- Output: 120 x 120 pixels, each assigned one of 44 CORINE L3 classes

Conceptually:

```
Input: 12 x 120 x 120
      |
      M1
      |
44 x 120 x 120 logits
      |
      argmax
      |
120 x 120 class map
```

The document specifies a dual-encoder U-Net.

19. Why Dual Encoder?

Instead of throwing S1 + S2 into one encoder, the architecture does: S1 → Encoder 1, S2 → Encoder 2 → fusion → decoder, because Sentinel-1 and Sentinel-2 represent different physical modalities. Sentinel-1 (SAR) carries structure, roughness, and radar response information; Sentinel-2 (optical multispectral) carries vegetation, water, and spectral characteristics. The network learns separately from each modality and then combines them.

20. M1 Architecture

- Recommended encoder: **ConvNeXt-V2-Tiny** (alternative: ResNet-34)
- Decoder: U-Net decoder
- Fusion: S1 features + S2 features → concatenation → 1x1 convolution → Squeeze-and-Excitation gate

The SE gate learns which channels/features are more useful. The complete model is roughly 30–45 million parameters.

21. Why U-Net?

Because this is dense prediction — the model needs to know what class every pixel is. U-Net's encoder → compressed representation → decoder → pixel-level prediction structure, with skip connections, preserves spatial information. This matters because the architecture eventually needs

regions, boundaries, connected components, boxes, and centroids — a model that only understands the scene globally is insufficient.

22. 44-Class Output + Auxiliary Heads

The main output is 44 CLC L3 classes, but the architecture also proposes an auxiliary 19-class head and an auxiliary coarse-7 head, because the model can learn hierarchical representations (L3 detailed classes $\rightarrow$ L2/L1 relationships $\rightarrow$ 19-class representation $\rightarrow$ coarse-7 representation). This is useful because questions may refer to different hierarchy levels.

23. M1 Loss Function

$$L = L_{\text{CE}} + 0.5 * L_{\text{Lovasz}} + 0.3 * L_{\text{hier}} + 0.2 * L_{\text{scale}}$$

Cross entropy (L_{CE}): standard pixel classification loss; unclassified pixels are ignored. Class weights use inverse-square-root frequency, capped at 5x, because rare classes otherwise get almost no influence during training.

24. Lovász-Softmax

The second term ($0.5 \times L_{\text{Lovasz}}$) is designed to optimize a surrogate related to IoU. Since segmentation is evaluated heavily using IoU, the architecture wants the training objective to care about the same kind of metric. It is also expected to help rare classes.

25. Hierarchy-Aware Loss

Suppose the true class is forest and the prediction is another forest-related sibling class — that’s not as bad as forest $\rightarrow$ artificial surface. So the architecture weights these errors differently: crossing a Level-1 branch (Artificial $\leftrightarrow$ Forest) gets 1.5x, while sibling confusion gets 1.0x. The reason is that the benchmark’s adversarial questions deliberately exploit semantically similar classes.

26. Scale Consistency Loss

The architecture wants the model to handle changes in spatial scale: original image $\rightarrow$ resample 0.5x–4x $\rightarrow$ M1 $\rightarrow$ prediction $\rightarrow$ resample back $\rightarrow$ compare with original prediction, using KL divergence. The idea: the same scene should not produce radically different semantic interpretations simply because its scale changed. This matters because the eventual ISRO evaluation may involve imagery at much finer GSD than Sentinel-2.

27. Why M1 is Trained From Scratch

A controversial but deliberate decision: M1 = FROM SCRATCH rather than using a pretrained reBEN classification model. Reason: the training set has 229,114 labelled training patches, which the architecture argues is data-rich enough to learn the required spatial features directly. Also, a classification model is not necessarily ideal for precise pixel boundaries. The document explicitly says ImageNet initialization should be an ablation, not the primary assumption.

28. M1 Training Dataset

The primary dataset is **BigEarthNet v2 / reBEN**, containing Sentinel-1 + Sentinel-2 + CORINE reference maps: 549,488 total pairs, filtered to 464,044, with a train split of 229,114. Importantly, this is not simply image classification data — the architecture uses the pixel-level CORINE reference maps as segmentation supervision.

29. M2 — Symbolic Geometry Engine

Now we reach the brain of the deterministic reasoning layer. M2 is not a neural network. It uses `scipy.ndimage` and `skimage.measure`. Main operations: `label`, `binary_dilation`, `binary_opening`, `binary_fill_holes`, `regionprops`. Its job is to turn a segmentation map into measurable spatial facts.

30. M2 Step 1 — Hierarchy Aggregation

This is the new critical step. Suppose M1 predicts 44 L3 classes but the question asks for a higher-level category. The system uses a taxonomy YAML to map L3 $\rightarrow$ requested CLC level, then creates a binary mask. For example, continuous + discontinuous urban fabric + other artificial classes $\rightarrow$ one “artificial surfaces” binary mask. Only after this aggregation does connected-component analysis happen, which prevents one conceptual region from being counted as multiple regions simply because its L3 subclasses differ.

31. M2 Step 2 — Binary Mask

Once the requested class is known: target class $\rightarrow$ binary mask (1 = forest, 0 = everything else). Now geometry can operate on that mask.

32. M2 Step 3 — Morphological Cleanup

The architecture uses **opening** (removes tiny noisy structures) and **hole filling** (fills small holes inside regions), because segmentation predictions may contain small artifacts. The parameters are fitted, not guessed.

33. M2 Step 4 — Connected Components

This is how the system determines separate regions, using `scipy.ndimage.label()` to assign component IDs, then `count = number of components`.

34. Connectivity: 4 vs. 8

This is extremely important for counting. Under **4-connectivity**, pixels connect only through up/down/left/right. Under **8-connectivity**, diagonal pixels also connect. For example, a diagonal checkerboard pattern is two regions under 4-connectivity but one region under 8-connectivity. The architecture must determine which convention matches the benchmark by fitting 4 vs. 8 against ground-truth maps and released answers.

35. Minimum Mapping Unit (MMU)

Suppose segmentation creates a 3-pixel noise blob — should that count as a real region? Probably not. So the architecture introduces a Minimum Mapping Unit: any component below the fitted threshold is discarded. Again, this is fitted against benchmark conventions.

36. Region Properties

For every surviving component, M2 computes area, bounding box, centroid, and fill ratio using `regionprops`. This gives the system the raw geometric information needed for many tasks.

37–41. Geometric Task Computations

Presence: `number of components > 0`.

Count: number of connected components (e.g., forest components = 7 $\rightarrow$ answer = 7). Deterministic once the map and geometric conventions are fixed.

Area: `number of pixels x GSD2`. For 10m imagery, one pixel = 100 m². So 1,000 pixels x 100 m² = 100,000 m². The architecture then rounds to the nearest 1,000 m², a convention verified from the benchmark.

Adjacency: `dilate(A)  $\rightarrow$  intersect B`. If the intersection exists, `adjacent = TRUE`. The dilation radius `k` is fitted, because “adjacent” in the benchmark may not literally mean touching exactly at a pixel boundary — the system reverse-engineers the benchmark’s actual convention.

Relative position: compare region centroids (e.g., $\Delta x > 0$, $\Delta y < 0$ might correspond to northeast), using an 8-way compass (N, NE, E, SE, S, SW, W, NW). Another example of something the VLM doesn’t need to learn.

42–43. Referring Expression and Point Detection

The benchmark contains referring-expression tasks. The system filters candidate regions using `1% <= area <= 50%` of image and `bbox fill >= 40%`, then selects based on the query’s qualifier (largest, smallest, north-most, etc.). This makes referring detection a geometric selection problem rather than a generative vision problem.

For referring-point detection, given an input point (x, y), the system checks which connected component contains this point, then returns that component’s bounding box — nearly trivial once the segmentation exists.

44. Caption Attributes

The geometry engine can also generate structured attributes: class present, number of regions, area, region size, adjacency, tier. The tier system is: `>25%  $\rightarrow$  primary`, `5–25%  $\rightarrow$  secondary`, `<5%  $\rightarrow$  marginal`. These structured facts can then be given to the captioning system.

45. M3 — Binary VQA Decision Head

M3 solves YES/NO, especially for quantitative questions. It receives computed value + stated value + uncertainty information. Example: question “Is forest approximately 45,000 m²?” — geometry engine computes 48,000, question parser extracts stated = 45,000. M3 decides YES or NO and outputs P(yes).

46. Why Not Just Use a Fixed Tolerance?

Because the benchmark’s “NO” answers are deliberately generated as near misses. The correct tolerance isn’t necessarily $\pm 10\%$ or $\pm 5,000$ m². The decision boundary must be learned from the benchmark’s answer-generation behavior. So M3 is a small tabular model — recommended: LightGBM (~200 trees, depth = 4), or L2 logistic regression if maximum interpretability is desired.

47. M3 Features

Computed value, stated value, absolute difference, log ratio, rank difference, question subtype, class ID, class pixel share, mean segmentation margin, TTA count standard deviation, components near MMU, sibling confusion prior. M3 isn’t simply `computed > stated?` — it has information about how reliable the computed value is.

48. Why Train M3 on Predicted Maps?

Don’t train M3 using perfect ground-truth geometry — instead: M1 predicted map $\rightarrow$ M2 computed value $\rightarrow$ M3 training features. Why? Suppose M1 systematically over-segments forests (real = 10 regions, M1 = 12 regions). M3 can learn that this system tends to overestimate this kind of quantity, compensating for systematic segmentation bias.

49–51. M4 — MCQ Option Scorer

Suppose the question is “Which of the following is the area of forest?” with options A: 10,000, B: 20,000, C: 30,000, D: 40,000. M2 might calculate 31,200. M4 chooses the closest option using a fitted metric.

Why not simply absolute difference? Because different quantities need different scales — for area, 10,000 vs. 20,000 is very different from 100,000 vs. 110,000; relative/logarithmic distance makes more sense. For counts, 1 vs. 2 is a much bigger difference than 100 vs. 101. So M4 uses log scale for area, rank scale for count, and a fitted distance/temperature. The system calculates `score(option k) = -distance(computed, option k)`, then converts these into a four-option softmax.

M4 has two branches: **geometric MCQ** (presence, area, count, adjacency, relative position — uses the geometry result) and **metadata MCQ** (country, season, climate — uses M5).

52–55. M5 — Scene Metadata Classifier

M5 solves the three MCQ categories that geometry cannot solve: country, season, and Köppen climate zone. Model: **ConvNeXt-Tiny** with 12-channel input and 3 classification heads (one per

attribute).

Why does M5 exist? There is no connected-component calculation that tells you which country an image was captured in, or what season it was acquired in — geometry doesn’t help here, so a dedicated visual classifier is needed.

M5 dataset: reBEN metadata contains country, acquisition date, and geolocation. Köppen climate zone can be obtained from coordinates using a climate raster, giving labelled metadata for hundreds of thousands of examples. The model trains country $\rightarrow$ 10 classes, season $\rightarrow$ 4 classes, climate $\rightarrow$ k classes. For benchmark MCQ: model probabilities $\rightarrow$ restrict to the four offered options $\rightarrow$ argmax.

Important gate: if the benchmark’s input metadata already provides country, date, and geolocation, M5 may become unnecessary (e.g., geolocation $\rightarrow$ country lookup would be exact). The architecture says to inspect the benchmark parquet in week 1 before deciding whether M5 is needed.

56–59. M6 — Siamese Semantic Change Model

Required when there are images at T1 and T2, with a question like “What changed between these two images?” The model needs to understand what existed before, what exists now, and what changed.

Why Siamese? T1 $\rightarrow$ encoder, T2 $\rightarrow$ same encoder (shared weights). Both images represent the same type of scene at different times; shared weights force a common representation rather than one that varies simply due to date-specific differences.

M6 produces a semantic map for T1, a semantic map for T2, and a change mask. The semantic maps use the SECOND taxonomy (6 classes). The system then creates a transition matrix $T[i][j]$ describing how much changed from class i to class j.

Example transition matrix: Forest $\rightarrow$ Building = 500 pixels, Crop $\rightarrow$ Building = 800 pixels, Water $\rightarrow$ Crop = 100 pixels. Questions become calculations over this matrix — e.g., “What changed to buildings?” sums Forest $\rightarrow$ Building + Crop $\rightarrow$ Building + ... Again: image understanding $\rightarrow$ semantic maps $\rightarrow$ transition matrix $\rightarrow$ arithmetic $\rightarrow$ answer.

60. Why SECOND?

The architecture uses SECOND because the CDVQA benchmark itself was generated from SECOND semantic change maps. This is extremely important — it means the system again follows the benchmark’s own construction: SECOND maps $\rightarrow$ CDVQA questions. So the proposed system does image pair $\rightarrow$ M6 $\rightarrow$ SECOND-like semantic maps $\rightarrow$ transition matrix $\rightarrow$ CDVQA answer, repeating the same core strategy used in the main branch.

61. M6 Training

Unlike M1 (trained from scratch), M6 uses an ImageNet-pretrained encoder with full fine-tuning, because M6 has only around 2,968 public pairs and only around 1,600 pairs in the CDVQA training split — data-poor, so pretraining is valuable.

Model	Data	Strategy
M1	229k patches	From scratch
M6	~1.6k CDVQA training pairs	Transfer learning

62. M6 Loss

$L = \text{semantic_T1} + \text{semantic_T2} + \text{change} + 0.3 * \text{consistency}$

The consistency term ensures that if M1 class $\neq$ M2 class, change should generally be 1, and if they're equal, change should generally be 0 — helping the three outputs agree logically.

63–65. M7 — InternVL3-1B

Finally, the large language/vision model: **OpenGVLab/InternVL3-1B**. The architecture deliberately does not make this the central reasoning engine. Its role is free-form language, captioning, change narration, fallback, and compliance with the RS-adaptation requirement. It is explicitly not responsible for counting, area, adjacency, relative position, or boxes — those belong to the symbolic path.

M7 architecture follows the published remote-sensing adaptation structure: S1 $\rightarrow$ BEN-pretrained ViT $\rightarrow$ projection; S2 $\rightarrow$ BEN-pretrained ViT $\rightarrow$ projection; RGB $\rightarrow$ InternViT-300M; Text $\rightarrow$ tokenizer; everything $\rightarrow$ Qwen2.5-0.5B LLM with LoRA. Only about 5.8M parameters are trainable out of approximately 1.1B.

Why LoRA for M7? Because M7 is around 1.1 billion parameters and full fine-tuning is expensive. LoRA modifies a small low-rank portion of the model ($r = 8$, $\alpha = 32$, $\text{dropout} = 0.1$), making it feasible on a single consumer GPU.

66. M7 Training Curriculum

- **Step 1 — Projection warm-up** (~2,000 steps, LoRA frozen): train only modality projections, because S1/S2 visual tokens are new inputs that must be mapped into a representation space the LLM understands.
- **Step 2:** train LoRA + projections using captioning, since captions provide dense language supervision.
- **Step 3:** optional change-narration adapter.
- **Step 4:** don't train VQA/MCQ/grounding adapters unless symbolic performance fails.

This is how the architecture saves enormous compute.

67. M7 Decoding Strategy

The VLM is not allowed to freely generate answers for tasks that can be constrained:

- **Binary:** compare `logit("yes")` vs. `logit("no")` — no free generation.
- **MCQ:** calculate log-likelihood for each option, normalize for length, choose the highest.
- **Bounding box:** grammar-constrained decoding.
- **Caption:** free generation is allowed.

This is another major reliability mechanism.

68. A Note on the Published 34.04 BLEU

The architecture explicitly warns: don't assume you will reproduce 34.04 BLEU-4. The published result used 4x H200 GPUs, approximately 2 days, and per-task fine-tuning. The student system has far less compute, so the fair comparison is InternVL3-1B zero-shot vs. your adapted InternVL3-1B, rather than pretending you reproduced the exact published training setup.

69–72. M8 — Template-to-Style Caption Rewriter

This model is conditional. It exists because the benchmark's reference captions aren't simple templates — they were: template → Llama-4-Scout-17B paraphrasing → self-refinement → reference caption. So if your geometry engine produces “The image contains forest and water,” the benchmark reference might use a completely different sentence structure, and BLEU-4 cares about word sequence overlap. Therefore raw symbolic templates may have low BLEU despite being factually correct — that's why M8 exists.

M8 model: recommended Flan-T5-base (250M) or LoRA on Qwen2.5-0.5B-Instruct. It takes the exact template and produces a dataset-style caption. It does not see the image — the factual content has already been determined by the geometry engine.

Factuality safeguard: after rewriting, the generated caption is re-extracted for attributes and compared against the source structure. If the facts don't match, it's rejected — preventing “better BLEU” from being achieved through hallucination.

M8 gate (decided in week 2): ground-truth map → template caption → BLEU/ROUGE/METEOR/CIDEr. If BLEU ≥ 35 , skip M8 (templates are good enough). If $10 \leq \text{BLEU} < 35$, build M8. If BLEU < 10 , don't waste time — use M7 for captioning.

73–75. M9 — Confidence Calibrator

M9 estimates $P(\text{answer is correct})$, which is different from raw model confidence — a confident segmentation can still produce a wrong count due to merging two regions. M9 works at the answer level.

Model: L2 logistic regression + isotonic regression. Inputs include mean class margin, TTA answer stability, component-count standard deviation, area interval width, minimum component margin, band presence, and sibling confusion prior. Output: $P(\text{correct})$ and a high/medium/low confidence band.

TTA (Test-Time Augmentation): the architecture runs the same image under 4 rotations x 2 flips = 8 transformations, reverses the transformations, and averages the logits. If all eight produce the same answer, that's stable; if they vary widely, that's unstable — this instability becomes a confidence signal. TTA therefore both improves prediction and estimates uncertainty.

76–77. Stage 8 — Abstention

M9 determines whether the symbolic answer is trustworthy. There are two modes:

- **Operational mode:** if $P(\text{correct})$ is below threshold, the system can abstain and identify the likely problem.
- **Benchmark mode:** cannot abstain (abstention = wrong answer for leaderboard scoring), so it falls back through symbolic $\rightarrow$ M7 VLM $\rightarrow$ class prior $\rightarrow$ majority.

The fallback chain is a cascade based on confidence level (high $\rightarrow$ answer; medium $\rightarrow$ answer + uncertainty; low $\rightarrow$ abstain in operational mode, or fall back through M7/prior/majority in benchmark mode). Each task has a defined producer; M9 simply determines whether to trust it — it is not a simple ensemble vote across M1/M5/M6/M7.

78. Stage 9 — Answer Assembler

The most important rule: **a language model may phrase the answer, but it may never produce the numerical value.** If geometry calculates area = 42,000 m², the assembler inserts 42,000 m² into the answer template. The VLM cannot decide “maybe 45,000” — that would destroy auditability. This is one of the strongest architectural rules.

79. Final Outputs

1. **Structured JSON:** value, unit, confidence, evidence references.
2. **Evidence:** mask overlay, component contours, bounding boxes, area table, margin heatmap, GeoJSON, GeoTIFF.
3. **Execution trace:** task, tools, parameters, timings, model name, weight hash.
4. **PDF report:** question, input, map, arithmetic, confidence, trace, model hash, dataset attribution.

This means the system can explain why it gave the answer, not just give an answer.

80–81. How All Models Communicate

The models do not communicate using natural language. M1 $\rightarrow$ tensor/map; M5 $\rightarrow$ probability vectors; M6 $\rightarrow$ semantic maps; M2 $\rightarrow$ structured geometric data. Then structured data $\rightarrow$ M3/M4/M7. The architecture explicitly forbids M1 $\rightarrow$ sentence $\rightarrow$ M7 $\rightarrow$ sentence $\rightarrow$ M3, because errors compound. Instead: M1 $\rightarrow$ typed map, M2 $\rightarrow$ typed geometry, M3 $\rightarrow$ decision, M9 $\rightarrow$ confidence. This is a major reason the architecture is auditable.

The **number-flow rule:** Image $\rightarrow$ M1 $\rightarrow$ M2 $\rightarrow$ NUMBER. The number cannot flow backward through generated language. Never extract a number from a VLM sentence as the answer — that would make the VLM the numerical authority. The architecture wants geometry to remain the numerical authority.

82. Datasets — The Complete Picture

The architecture deliberately avoids collecting random remote-sensing datasets. Every dataset must answer one of five questions:

1. What does the LULC map look like? $\rightarrow$ reBEN, WorldCover India
2. What is the answer grammar? $\rightarrow$ BigEarthNet.txt

3. What does change look like? → SECOND, CDVQA
4. What does metadata look like? → reBEN metadata
5. Does the system work in India? → Sentinel-1/2 India, WorldCover, Dynamic World, Bhuvan

83. Dataset 1 — reBEN

The main dataset: Sentinel-1 + Sentinel-2 + CORINE L3 reference maps. 549,488 pairs; filtered to 464,044; train 229,114; validation 118,095; test 116,835. The primary target is 44-class CLC L3 — this is M1’s supervision.

84. Dataset 2 — BigEarthNet.txt

Contains the benchmark’s captions, binary VQA, MCQ, referring expressions, and referring-point tasks — about 9.6M annotations across the larger dataset. The actual benchmark evaluation split is 1,082 image pairs and 15,029 annotations, including 6,927 binary, 5,550 MCQ, 970 captions, and 1,582 referring. The benchmark split must be quarantined until final evaluation.

85–86. Datasets 3 & 4 — SECOND and CDVQA

SECOND is used for M6. It contains bi-temporal imagery and semantic change labels, publicly available as 2,968 pairs with 6 semantic classes. It was chosen because it is the source behind CDVQA’s generated questions.

CDVQA is the change VQA benchmark: >122k QA pairs, with 65,967 QA / 1,600 train pairs, 16,441 validation QA / 400 validation pairs, and two official test splits. Answers come from a closed set of 19 categories. The system evaluates M6 + transition-matrix reasoning on this benchmark.

87–88. Datasets 5 & 6 — VRSBench and RSVQA

VRSBench is used for mandated evaluation and scale robustness: 29,614 images, 52,472 referring expressions, 123,221 QA. The architecture explicitly says not to use its taxonomy as CORINE class supervision — it can instead help with scale-consistency experiments.

RSVQA is used for evaluation only, not training, because its taxonomy and question distribution are different — the architecture deliberately avoids adding unnecessary training noise.

89–92. Indian Data

Sentinel-1 + Sentinel-2: the key Indian adaptation dataset. The architecture uses exactly the same sensors as the European training data (10m), so the difference becomes geography rather than sensor + geography + resolution + preprocessing — a scientifically cleaner experiment.

WorldCover: ESA WorldCover 10m provides 11 land-cover classes, used as weak supervision, not perfect ground truth.

Dynamic World: provides 10m land cover plus per-class probabilities, useful for identifying high-confidence vs. uncertain pixels, allowing the training loss to be weighted accordingly.

Bhuvan: provides ISRO/NRSC land-cover information, used primarily for coarse validation, class-prior agreement, and an independent cross-check — not as equivalent to 10m CORINE pixel labels.

93–95. BHARAT-EO, BHARAT-TRAIN, BHARAT-VAL

The architecture proposes building an Indian corpus (BHARAT-EO) sampling across agro-climatic zones, seasons, urban, peri-urban, rural, forest, coastal, and arid regions, with rare classes deliberately represented.

BHARAT-TRAIN is used for Indian adaptation. **BHARAT-VAL** is used only for final Indian evaluation — never trained on, never tuned on, never used for early stopping. This is exactly how you prevent accidentally overfitting the Indian result. The split is geographic rather than random.

Because CORINE and WorldCover have different taxonomies, comparing them directly would be scientifically invalid. Both are instead mapped to **coarse-7** (built-up, cropland, tree cover, grassland/shrub, water, bare/sparse, wetland), making the Europe vs. India comparison meaningful.

96. Indian Domain Adaptation

- **Stage 1:** re-estimate normalization statistics, no gradient, cheap.
- **Stage 2:** freeze encoder, train decoder + head on Indian weak labels.
- **Stage 3:** if still improving, unfreeze last encoder stage with a very low learning rate.
- **Stage 4:** replay 20–30% European batches to prevent catastrophic forgetting; if European mIoU drops more than ~2 points, stop/back off.

97. Why Not LoRA for M1?

Because M1 is only 30–45M parameters, full fine-tuning is feasible. LoRA is useful primarily when the model is huge and full fine-tuning is expensive — that’s M7. M1 doesn’t have that problem, so: M1 → full/staged fine-tuning; M7 → LoRA.

98. Why India is the Strongest Argument for This Architecture

VLM-only: India image → VLM. To adapt it properly you’d want Indian image + Indian text instructions + Indian answers, but there isn’t an equivalent Indian BigEarthNet.txt corpus, so adaptation is difficult.

SatQuery hybrid: India image → M1 segmentation → geometry → answer. Indian pixel labels can be obtained, so the architecture can adapt the segmenter, and after that the geometry engine doesn’t care whether the pixels came from Bavaria or Bengaluru — the same mathematical operation works. This is one of the strongest scientific arguments in the whole document.

99–100. Data Augmentation Strategy

- **Geometric:** horizontal flip, vertical flip, rotations — because land cover doesn’t have a fixed orientation.
- **Band dropout:** simulates reduced sensor spectral information, particularly keeping B02/B03/B04/B08 to approximate a four-band optical configuration.

- **Modality dropout:** sometimes remove S1 or S2 entirely, to teach robustness.
- **Scale jitter:** 0.5x–4x, simulating GSD changes.
- **SAR speckle:** simulates different radar statistics.
- **Radiometric jitter:** simulates sensor calibration differences.

Why no Mixup or CutMix? Because this project relies on connected regions, and mixing two images can create artificial boundaries that change the number of components, corrupting the geometry logic. So the architecture intentionally excludes Mixup and CutMix — an important domain-specific design choice.

101. Class Balancing

M1 uses four mechanisms: (1) inverse-square-root weighting ($1/\sqrt{\text{frequency}}$), capped at 5x instead of $1/\text{frequency}$, which could become enormous; (2) Lovász loss to help optimize IoU; (3) rare-class oversampling around 2–3x, not 10x; (4) per-class IoU reporting, because overall mIoU can hide poor performance on important classes.

102–103. Validation Methodology

The architecture rejects a random train/validation split because neighboring satellite patches are spatially correlated. Instead: geographic block cross-validation with $k = 5$, using country blocks or 1° grid cells, then a final evaluation using a held-out geographic split.

Why spatial leakage is dangerous: if patches from the same region end up split across training and validation, the model may see nearly the same landscape in both sets, producing an inflated mIoU that doesn’t reflect real geographic generalization. The architecture wants validation to simulate “what happens when we go somewhere the model hasn’t seen?”

104–106. Accuracy is Not Guessed

The architecture explicitly refuses to claim a target like “we will get 85%.” Instead it defines:

$$\text{TARGET}(t) = \text{ORACLE}(t) \times \text{TRANSFER}(t)$$

Oracle accuracy: ground-truth map $\rightarrow$ M2 geometry engine $\rightarrow$ answer, with no segmentation error — the maximum accuracy possible from the symbolic method. If oracle is only 60%, improving M1 won’t solve the problem; the geometry/answer-generation assumptions are wrong.

Transfer factor: predicted map $\rightarrow$ M2 $\rightarrow$ accuracy. If oracle = 95% and predicted = 76%, transfer ≈ 0.80 — meaning 20% of the achievable symbolic performance is lost because of segmentation. High oracle + low transfer $\rightarrow$ improve M1. Low oracle $\rightarrow$ fix symbolic/answer grammar. This is a very strong experimental design.

107–108. Error Analysis

Every error should be categorized: was segmentation wrong? hierarchy mapping wrong? MMU wrong? the M3 decision boundary wrong? a parsing failure? This tells the team where the next improvement should come from.

Count-error decomposition: counting can fail in two opposite ways — **over-counting** (one real region fragmented into multiple predicted components, fixable via opening) or **under-counting** (two real regions incorrectly connected into one predicted component, fixable by removing false bridges). These errors should be analyzed separately.

109–111. Baselines and Statistical Significance

Compare against blind (question text only), majority (most common answer), and class prior (marginal land-cover distribution) baselines, alongside VLM-only, symbolic-only, hybrid, and oracle systems. This gives a credible scientific comparison answering “does our architecture actually work?” rather than “does our architecture sound good?”

Because the benchmark is relatively small, small accuracy differences (e.g., 72% vs. 74%) cannot automatically be claimed as meaningful. The architecture recommends **bootstrap confidence intervals** (resampled at the image-pair level, not annotation level, since annotations within the same pair are correlated) and **McNemar’s test** for paired comparisons. It recommends not treating differences smaller than roughly 3 points (binary) or 4 points (MCQ) as meaningful without stronger evidence.

112–113. Architecture A vs. B vs. C

- **Architecture A** — the previous complex system: many components, many datasets, many experimental branches. High risk.
- **Architecture B** — recommended: same core idea + taxonomy fixed + hierarchy + M3 + M4 + M5 + M9 + Indian evaluation, minus unnecessary components.
- **Architecture C** — a simple VLM baseline (InternVL3-1B + LoRA + BigEarthNet.txt), very easy to build but weak on core symbolic tasks and Indian adaptation.

The document recommends Architecture B, which cuts: cross-modal attribution as a runtime component, the caption stylizer unless the gate says build it, LEVIR-CD, LEVIR-CC, LoveDA, NISAR, a 0.5B query LLM, test-time adaptation by default, multi-scale training, and unnecessary frontend polish. It says never cut: the symbolic path, evaluation harness, execution trace, input validation, oracle experiment, BHARAT-VAL, and mandatory capabilities.

114. Final Implementation Blueprint

STEP 0: DATASETS

|

STEP 1: PREPROCESSING

|

MODEL 1: M1 SEGMENTER

|

MODEL 2: M2 GEOMETRY

|

MODEL 3+4: BINARY + MCQ

|

MODEL 5: METADATA

|

MODEL 6: CHANGE
 |
 MODEL 7: VLM
 |
 MODEL 8: CAPTION STYLIZER (if required)
 |
 MODEL 9: CONFIDENCE
 |
 ANSWER ASSEMBLER

115. Exact Model Summary

ID	Component	Model/Method	Purpose	Training
M1	LULC segmentation	Dual-encoder U-Net + ConvNeXt-V2-Tiny	44-class pixel map	From scratch
M2	Geometry	scipy + skimage	Exact spatial measurements	Deterministic
M3	Binary decision	LightGBM / Logistic Regression	YES/NO	From scratch
M4	MCQ	Fitted distance + softmax	Choose option	Fit parameters
M5	Metadata	ConvNeXt-Tiny	Country/season/climate	Transfer learning
M6	Change	Siamese U-Net	Semantic change	Transfer learning
M7	VLM	InternVL3-1B + LoRA	Language/caption/fallback	1.6B
M8	Caption style	Flan-T5-base / Qwen LoRA	Dataset-style caption	Conditional fine-tune
M9	Confidence	Logistic + isotonic	P(correct)	Calibration
M10	Query parser fallback	TF-IDF + Linear SVM	8-way intent classification	From scratch

116. Pretraining Strategy — Memorize This Distinction

- **M1:** 229k labelled patches → data-rich → from scratch.
- **M5:** scene classification, ImageNet features useful → transfer learning.
- **M6:** ~1.6k training pairs → data-poor → ImageNet transfer.
- **M7:** 1.1B parameters, full fine-tuning expensive → LoRA.
- **M8:** 250M text model, many aligned pairs → fine-tune.
- **M3/M4/M9/M10:** small tabular/classification problems → train from scratch / fit.

This is a very defensible design choice.

117. The 8-Week Execution Plan

- **Week 1:** build data pipeline, parquet parser, taxonomy YAML, evaluation skeleton, licence audit. Deliverable: `ANSWER_GRAMMAR.md` — CLC level, metadata available, distractor spacing.
- **Week 2:** build geometry engine; fit connectivity, MMU, dilation; run oracle experiment, caption oracle, blind baseline, majority baseline. Deliverable: oracle accuracy and oracle BLEU.
- **Week 3:** train M1 candidates (U-Net, SegFormer, etc.); measure mIoU + downstream symbolic accuracy. Deliverable: transfer factor.
- **Week 4:** run the crucial A1 experiment (blind vs. VLM vs. symbolic vs. oracle); build M3, M4, M5. Deliverable: a falsification test — if symbolic isn’t working, stop investing heavily and use the VLM fallback.
- **Week 5:** build M6, CDVQA conversion, M9, frontend endpoints; refit symbolic parameters on predicted maps. Deliverable: CDVQA accuracy + ECE + risk-coverage.
- **Week 6:** build M7, M8 if required, Indian evaluation, Indian adaptation. Deliverable: Europe result and India result side by side.
- **Week 7:** ablations, perturbation tests, bootstrap CIs, McNemar tests, error analysis, PDF report.
- **Week 8:** freeze — no new features, only rehearse, finalize results, limitations, and presentation.

Uncontrolled feature development is identified as one of the largest project risks.

118. The Three Numbers That Decide Whether the Project is Working

1. **End of Week 2 — Oracle symbolic accuracy:** “If segmentation were perfect, would our approach work?”
2. **End of Week 3 — Transfer factor:** “How much of that theoretical performance survives imperfect segmentation?”
3. **End of Week 6 — BHARAT-VAL coarse-7 mIoU:** “Does our approach actually transfer to India?”

These are much more scientifically useful than inventing an arbitrary target like 85% before running experiments.

119. Published Reference Numbers

Model	Caption BLEU-4	Binary VQA	MCQ	Ref-exp
Best RS-specific	1.66	58.38	35.26	16.18
Best general CV	0.96	61.96	37.55	31.73
InternVL3-1B zero-shot	0.45	54.11	26.76	5.76
RS-InternVL fine-tuned	34.04	73.29	51.49	65.84

The architecture is careful about these numbers: the 34.04 result involved multiple task-specific adapters and significant H200 compute, so a one-GPU student system should not claim it will reproduce 34.04.

120. Why Adjacency is Such a Powerful Demonstration

Published: a ~2T parameter model achieves only 55.86% on adjacency. Your architecture: M1 segmentation → binary mask → binary_dilation → intersection → exact adjacency. The argument isn't "our smaller neural network is smarter than a 2T model" — it's: "we changed the problem formulation. Once perception is separated from deterministic spatial measurement, some tasks no longer need generative reasoning at all." That's a much stronger scientific argument.

121–122. The Two Biggest Limitations

M1 segmentation quality. If M1 is wrong, M2 is wrong, and the answer is wrong — the geometry engine can't recover information that wasn't correctly segmented. The architecture doesn't claim symbolic reasoning solves everything; the oracle tells us the symbolic ceiling and transfer tells us how much segmentation hurts.

The Indian sensor gap. Hidden ISRO/SAC imagery may use Cartosat-2S or RISAT with different spatial resolution, spectral bands, and SAR characteristics, and the team doesn't have equivalent hidden data. The architecture explicitly says never to fabricate a Cartosat/RISAT accuracy number — instead simulate band dropout, scale changes, speckle, and radiometric differences, and directly measure geographic transfer using Sentinel-1/2 India.

123. What Makes Architecture B Convincing

1. **Ground truth alignment:** the benchmark answers were generated from spatial reference maps, so predict-map-then-calculate-answer matches the benchmark construction.
2. **Deterministic measurements:** area, count, adjacency, and position don't need generative language — they can be calculated.
3. **Auditability:** every number has a derivation (pixels → component → area → answer) instead of "the VLM says 42,000."
4. **Indian adaptability:** the domain shift can be concentrated into M1, where Indian pixel supervision is obtainable.
5. **Compute efficiency:** only the parts that actually need deep learning get expensive models; everything else is rules, geometry, and small classifiers.

124–129. End-to-End Examples

Example — Area. "What is the area of forest in this satellite image?" GeoTIFF enters → V1 validates → preprocessing (S1 dB-normalized, S2 resampled-normalized) → query parser (`intent = AREA`, `class = FOREST`) → router (`AREA` → M1 → M2 → M9 → assembler) → M1 produces the 44-class map → M2 finds the forest class, builds a binary mask, cleans it up, and counts connected components/pixels → $\text{area} = \text{pixel count} \times \text{GSD}^2$ (e.g., 420,000 m²) → M9 computes P(correct) from segmentation margin, TTA stability, and component stability → assembler outputs value = 420,000, unit = m², plus confidence → final output includes the forest mask, component outlines, area table, confidence, and execution trace.

Example — Adjacency. "Are forest and water adjacent?" → Q1 (`intent = adjacency`, `classes = forest, water`) → R1 → M1 → 44-class map → M2 builds forest and water masks,

dilates forest, intersects with water → YES/NO → M3 if the benchmark’s binary decision format requires it → M9 → answer. M7 doesn’t need to answer the question at all.

Example — MCQ. “Which area is closest to the forest area?” (options A–D) → M1 → M2 computes 31,500 → M4 measures distance to each option → chooses option C → M9 → final answer. No language generation.

Example — Country. “Which country was this image acquired in?” M1/M2 are irrelevant here → image → M5 → country probabilities → restrict to the four offered options → argmax → M9 → answer.

Example — Caption. “Describe the image.” → M1 → M2 → structured attributes → caption template → M8 if required, or M7 → caption. Facts originate from M2; language style comes from M7/M8.

Example — Change. Image T1 + Image T2 + question → M6 produces semantic maps for T1 and T2 plus a change mask → transition matrix → M2 computes the change quantity → M3/M4 → M9 → answer. Again: perception → measurement → answer.

130. The Architecture in One Sentence

Learn what must be learned, compute what can be computed, concentrate the domain gap into the one model you can actually adapt, and measure everything — including the things that did not work.

That is essentially the entire SatQuery architecture.

131. If Your Team Asks “Why So Many Models?”

Don’t say “because we want a complex AI system.” Say: each model has one clearly bounded responsibility. M1 perceives land cover, M2 measures spatial structure, M3 decides binary tolerance, M4 selects MCQ options, M5 handles metadata, M6 handles temporal change, M7 handles language, M8 optionally matches caption style, M9 estimates answer reliability, and M10 handles query classification when rules fail. The models don’t compete with each other; they form a controlled cascade.

132. Fifteen Facts to Memorize

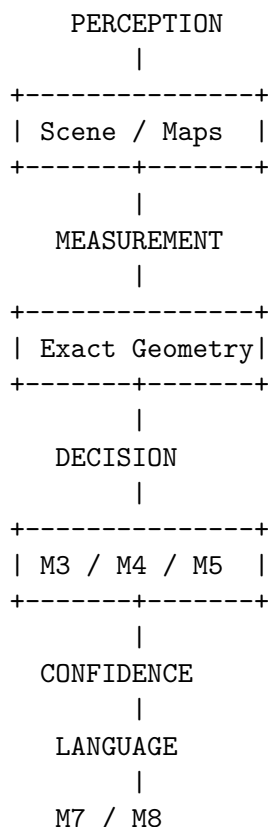
1. Main segmentation taxonomy = CORINE Level-3, 44 classes.
2. The 19-class representation is not the pixel-level segmentation target.
3. M1 = dual-encoder U-Net + ConvNeXt-V2-Tiny.
4. M1 input = Sentinel-1 (2 bands) + Sentinel-2 (10 bands).
5. M1 output = 44-class 120x120 land-cover map.
6. M2 = deterministic scipy/skimage geometry engine.
7. M2 computes presence, count, area, adjacency, relative position, referring box/point.
8. M3 = binary decision head.
9. M4 = MCQ option scorer.
10. M5 = country/season/climate classifier.
11. M6 = Siamese U-Net for semantic change + transition matrix.

12. M7 = InternVL3-1B + LoRA, mainly language/fallback.
13. M8 = optional caption style rewriter.
14. M9 = calibrated answer confidence.
15. Indian adaptation focuses primarily on M1, using BHARAT-EO/BHARAT-VAL.

133. The Most Important Conceptual Distinction

Never confuse these three things:

- **Perception** — what is in the image? Handled by M1, M5, M6, and M7 when necessary.
- **Measurement/reasoning** — what does the image mathematically imply? Handled by M2.
- **Language** — how do I express the result? Handled by M7/M8.



This is the core mental model to use when explaining the entire architecture. Importantly, the document itself does not claim that all of its expected performance numbers are established facts — published results are distinguished from engineering priors, and the architecture deliberately replaces guessed targets with the Week-2 oracle, Week-3 transfer factor, and Week-6 Indian evaluation.